

NON-UNIFORM MEMORY ACCESS (NUMA) ARCHITECTURE PLATFORMS

Systems employing a Non Uniform Memory Access (NUMA) architecture contain collections of hardware resources including processors, memory, and I/O buses, that comprise what is commonly known as a “NUMA node”. Two or more NUMA nodes are linked to each other via a high-speed interconnect. Processor accesses to memory or I/O resources within the local NUMA node are generally faster than processor accesses to memory or I/O resources outside of the local NUMA node, accessed via the node interconnect. ACPI defines interfaces that allow the platform to convey NUMA node topology information to OSPM both statically at boot time and dynamically at run time as resources are added or removed from the system.

In addition, devices such as coherent accelerators, coherent memory devices, or coherent switches can describe their NUMA characteristics information to OSPM or to system firmware using the Coherent Device Attribute Table (CDAT) structures. For more information, see the CDAT reference link at <http://uefi.org/acpi>, under the heading “Coherent Device Attribute Table (CDAT) Specification”.

17.1 NUMA Node

A conceptual model for a node in a NUMA configuration may contain one or more of the following components:

- Processor
- Memory
- I/O Resources
- Networking, Storage
- Chipset

The components defined as part of the model are intended to represent all possible components of a NUMA node. A specific node in an implementation of a NUMA platform may not provide all of these components. At a minimum, each node must have a chipset with an interface to the interconnect between nodes.

The defining characteristic of a NUMA system is a coherent global memory and/or I/O address space that can be accessed by all of the processors. Hence, at least one node must have memory, at least one node must have I/O resources, and at least one node must have processors. Other than the chipset, which must have components present on every node, each is implementation dependent. In the ACPI namespace, NUMA nodes are described as module devices. See the *Module Device* section.

17.2 System Locality

A collection of components that are presented to OSPM as a Symmetrical Multi-Processing (SMP) unit belong to the same System Locality, also known as a Proximity Domain. The granularity of a System Locality is typically at the NUMA Node level although the granularity can also be at the sub-NUMA node level or the processor, memory and host bridge level.

A System Locality is reported to the OSPM using Proximity Domain entries in the System Resource Affinity Table (SRAT), or using *_PXM (Proximity)* methods in the ACPI namespace. If OSPM only needs to know a near/far distinction among the System Localities, comparing Proximity Domain values is sufficient. See the *System Resource Affinity Table (SRAT)* and *_PXM (Proximity)* sections for more information.

OSPM makes no assumptions about the proximity or nearness of different proximity domains. The difference between two integers representing separate proximity domains does not imply distance between the proximity domains (in other words, proximity domain 1 is not assumed to be closer to proximity domain 0 than proximity domain 6).

17.2.1 System Resource Affinity Table Definition

The optional *System Resource Affinity Table (SRAT)* provides the boot time description of the processor and memory ranges belonging to a system locality. OSPM will consume the SRAT only at boot time. For any devices not in the SRAT, OSPM should use *_PXM (Proximity)* for them or their ancestors that are hot-added into the system after boot up.

The SRAT describes the system locality that all processors and memory present in a system belong to at system boot. This includes memory that can be hot-added (that is memory that can be added to the system while it is running, without requiring a reboot). OSPM can use this information to optimize the performance of NUMA architecture systems. For example, OSPM could utilize this information to optimize allocation of memory resources and the scheduling of software threads.

17.2.2 System Resource Affinity Update

Dynamic migration of the devices may cause the relative system resource affinity information (if the optional SRAT is present) to change. If this occurs, the System Resource Affinity Update notification (Notify event of type 0x0D) may be generated by the platform to a device at a point on the device tree that represents a System Resource Affinity. This indicates to OSPM to invoke the *_PXM (Proximity)* object of the notified device to update the resource affinity.

17.3 System Locality Distance Information

Optionally, OSPM may further optimize a NUMA architecture system using information about the relative memory latency distances among the System Localities. This may be useful if the distance between multiple system localities is significantly different. In this case, a simple near/far distinction may be insufficient. This information is contained in the optional *System Locality information Table*, and is returned from the evaluation of the *_SLI* object.

The SLIT is a matrix that describes the relative distances between all System Localities. To include devices that are not in the System Resource Affinity Table (SRAT), support for the *_PXM* object is required. The Proximity Domain values from SRAT, or the values returned by the *_PXM* objects are used as the row and column indices of the matrix.

Implementation Note: The size of the SLIT is determined by the largest Proximity Domain value used in the system. Hence, to minimize the size of the SLIT, the Proximity Domain values assigned by the system firmware should be in the range 0, ..., N-1, where N is the number of System Localities. If Proximity Domain values are not packed into this range, the SLIT will still work, but more memory will have to be allocated to store the “Entries” portion of the SLIT for the matrix.

The static SLIT table provides the boot time description of the relative distances among all System Localities. For hot-added devices and dynamic reconfiguration of the system localities, the `_SLI` object must be used for runtime update.

The `_SLI` method is an optional object that provides the runtime update of the relative distances from the System Locality *i* to all other System Localities in the system. Since `_SLI` method is providing additional relative distance information among System Localities, if implemented, it is provided alongside with the `_PXM` method.

17.3.1 Online Hot Plug

In the case of online device addition, the Bus Check notification (0x0) is performed on a device object to indicate to OSPM that it needs to perform the Plug and Play re-enumeration operation on the device tree starting from the point where it has been notified. OSPM needs to evaluate all `_PXM` objects associated with the added devices, and `_SLI` objects if the SLIT is present.

In the case of online device removal, OSPM needs to perform the Plug and Play ejection operation when it receives the Eject Request notification (0x03). OSPM needs to remove the relative distance information from its internal data structure for the removed devices.

17.3.2 Impact to Existing Localities

Dynamic reconfiguration of the system may cause the relative distance information (if the optional SLIT is present) to become stale. If this occurs, the “System Locality Information Update” notification (Notify event of type 0xB) may be generated by the platform to a device at a point on the device tree that represents a System Locality. This indicates to OSPM that it needs to invoke the `_SLI` objects associated with the System Localities on the device tree starting from the point where it has been notified.

17.4 Heterogeneous Memory Attributes Information

Optionally, OSPM may further optimize a NUMA architecture system using the Heterogeneous Memory Attributes. This may be useful if the memory latency and bandwidth attributes between system localities is significantly different. In this case, the information is contained in the optional Heterogeneous Memory Attributes (HMAT) and is returned from the evaluation of the `_HMA` object.

The HMAT structure describes the latency and bandwidth information between memory access Initiator and memory Target System Localities. System Locality proximity domain identifiers, as defined by Proximity Domain entries in the System Resource Affinity Table (SRAT), or as returned by `_PXM` object, are used in the HMAT structure.

Implementation Note: The size of the HMAT table is determined by number of memory initiator System Localities and the memory target System Localities. The static HMAT table provides the boot time description of the memory latency and bandwidth among all memory access Initiator and memory Target System Localities. For hot-added devices and dynamic reconfiguration of the system localities, the `_HMA` object must be used for runtime update.

The `_HMA` method is an optional object that provides the runtime update of the latency and bandwidth from the memory access Initiator System Locality “*i*” to all other memory Target System Localities “*j*” in the system.

Since `_HMA` method is providing additional memory latency and bandwidth information among System Localities, if implemented, it is provided alongside with the `_PXM` method.

17.4.1 Online Hot Plug

In the case of online device addition, the “Bus Check” notification (0x0) is performed on a device object to indicate to OSPM that it needs to perform the Plug and Play re-enumeration operation on the device tree starting from the point where it has been notified. OSPM needs to evaluate all `_PXM` objects associated with the added devices, and `_HMA` objects if the HMAT is present.

In the case of online device removal, OSPM needs to perform the Plug and Play ejection operation when it receives the “Eject Request” notification (0x03). OSPM needs to remove the ejected System Localities related information from its internal data structure for the removed devices.

17.4.2 Impact to Existing Localities

Dynamic reconfiguration of the system may cause the memory latency and bandwidth information (if the optional HMAT is present) to become stale. If this occurs, the Heterogeneous Memory Attributes Update notification (Notify event of type 0xE) may be generated by the platform to a device at a point on the device tree that represents a System Locality. This indicates to OSPM that it needs to invoke the `_HMA` objects associated with the System Localities on the device tree starting from the point where it has been notified.